

COMPLETE MASTER GUIDE

HTML

& Semantic HTML

From zero to a solid, professional foundation

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

THIS GUIDE COVERS

PART 1 — HTML Foundations: What HTML is, how it works, and the essential building blocks

PART 2 — Core HTML Elements: Text, links, images, lists, tables, forms, media, and more

PART 3 — HTML Attributes: Global attributes, event attributes, and element-specific attributes

PART 4 — Semantic HTML: Meaning-first markup, every semantic element explained in depth

PART 5 — Real-World Page Structure: Building a complete, semantic webpage layout

PART 6 — HTML Best Practices & Accessibility: Writing professional, inclusive HTML

PART 7 — Comprehensive Quiz: 20 multiple choice + 10 short answer + 2 coding challenges

Table of Contents

- Table of Contents 2
- PART 1 — HTML Foundations 4
 - 1.1 What Is HTML? 4
 - 1.2 A Brief History of HTML 4
 - 1.3 How HTML Works — The Browser's Job 4
 - 1.4 The Anatomy of an HTML Element 5
 - 1.5 The Standard HTML Document Structure 5
- PART 2 — Core HTML Elements 7
 - 2.1 Headings — <h1> to <h6> 7
 - 2.2 Paragraphs & Text Formatting 7
 - 2.3 Links — <a> 8
 - 2.4 Images — 9
 - 2.5 Lists 9
 - 2.6 Tables 10
 - 2.7 Forms — The Complete Reference 12
 - Input Types Reference 13
 - 2.8 Media Elements — Video, Audio & Canvas 14
- PART 3 — HTML Attributes 15
 - 3.1 Global Attributes 15
 - 3.2 Element-Specific Attributes 16
 - Input & Form Attributes 16
 - Link & Image Attributes 16
- PART 4 — Semantic HTML 18
 - 4.1 What Is Semantic HTML? 18
 - 4.2 Non-Semantic vs Semantic — Side by Side 18
 - 4.3 Page-Level Semantic Elements 19
 - <header> — Page or Section Header 19
 - <nav> — Navigation 19
 - <main> — Main Content 20
 - <section> — Thematic Section 20
 - <article> — Self-Contained Content 21
 - <aside> — Complementary Content 21
 - <footer> — Page or Section Footer 22
 - 4.4 Inline Semantic Text Elements 23
 - 4.5 Structural Semantic Elements 23
 - <figure> and <figcaption> 23

<details> and <summary>.....	24
<dialog>.....	24
<progress> and <meter>	25
4.6 Complete Semantic Elements Reference Table	25
PART 5 — Real-World Semantic Page Structure.....	27
PART 6 — HTML Best Practices & Accessibility.....	30
6.1 HTML Best Practices.....	30
6.2 Accessibility (a11y) — Writing HTML for Everyone.....	30
ARIA — Accessible Rich Internet Applications.....	31
Keyboard Navigation.....	31
Colour & Contrast	31
PART 7 — Comprehensive Quiz.....	33
Section A — Multiple Choice (Circle the correct answer — 1 mark each).....	33
Section B — Short Answer (Write 3–5 sentences per answer).....	35
Section C — Coding Challenges (Write correct, well-structured HTML)	36
Section A — Answer Key	36

PART 1 — HTML Foundations

1.1 What Is HTML?

HTML — HyperText Markup Language — is the standard language used to create the structure of every webpage on the internet. Every website you have ever visited, from Google to YouTube to your school's website, is built on HTML at its core.

HTML is NOT a programming language. It is a markup language. This means it uses special tags to label and describe content — telling the browser: this is a heading, this is a paragraph, this is an image, this is a link.

SIMPLE ANALOGY

Think of a webpage like a building.

HTML is the concrete and bricks — the raw structure, the walls and rooms.

CSS is the paint, tiles, and decoration — how it looks.

JavaScript is the electricity and plumbing — the behaviour and interactivity.

You cannot have CSS or JavaScript without HTML first. HTML is always the foundation.

1.2 A Brief History of HTML

Year	Milestone
1991	Tim Berners-Lee invents HTML at CERN. The very first version had only 18 tags.
1995	HTML 2.0 — the first formal standard. Added forms and tables.
1997	HTML 3.2 — added scripting, applets, and more formatting.
1999	HTML 4.01 — the dominant version for many years. Introduced CSS separation.
2008	HTML5 draft released — a revolutionary update with semantic elements, video, audio, canvas.
2014	HTML5 becomes the official W3C standard. Still the current version we use today.
Today	HTML is a living standard maintained by WHATWG — continuously updated.

1.3 How HTML Works — The Browser's Job

When you open a webpage, here is exactly what happens:

1. Your browser sends a request to the server for the webpage.
2. The server responds by sending back an HTML file (e.g., index.html).
3. The browser reads the HTML from top to bottom — this is called parsing.
4. The browser builds a DOM (Document Object Model) — an internal map of all the elements.

5. The browser renders (displays) the page visually based on the DOM structure.
6. CSS styles and JavaScript behaviour are then applied on top of the HTML structure.

KEY TERM: DOM

The Document Object Model (DOM) is the browser's internal tree-structure representation of your HTML page. Every element in your HTML becomes a node in the DOM tree. CSS uses the DOM to apply styles. JavaScript uses the DOM to read and change content.

1.4 The Anatomy of an HTML Element

An HTML element has three parts:

```
Opening tag      Content      Closing tag
   ↓             ↓             ↓
<h1>      Hello, World!      </h1>
```

The complete element = Opening tag + Content + Closing tag

Part	Explanation
Opening Tag	Marks where the element begins. Written as: <tagname>
Content	The actual text, image, or nested elements inside the element.
Closing Tag	Marks where the element ends. Written as: </tagname> (note the slash /).
Attribute	Extra information added inside the opening tag: . Format: name="value".
Self-closing	Some elements have no content and no closing tag: , , <hr>, <input>, <meta>.

1.5 The Standard HTML Document Structure

Every valid HTML5 document must follow this exact structure:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="A beginner's guide to HTML">
    <title>My First Webpage</title>
    <link rel="stylesheet" href="style.css">
  </head>

  <body>
```

```
<h1>Hello, World!</h1>
<p>This is my first webpage.</p>

<script src="app.js"></script>
</body>

</html>
```

Element / Attribute	What It Does
<!DOCTYPE html>	Tells the browser this is an HTML5 document. Must be the very first line. Not a tag — a declaration.
<html lang="en">	The root element. Wraps everything. lang="en" declares the page language (important for accessibility & SEO).
<head>	Contains metadata — information ABOUT the page that is not displayed visually.
<meta charset>	Sets character encoding to UTF-8 — supports virtually every language and special character.
<meta name=viewport>	Controls how the page scales on mobile screens. Required for responsive design.
<meta name=description>	A short summary of the page shown in search engine results.
<title>	The text shown on the browser tab and in search engine results. Very important for SEO.
<link rel=stylesheet>	Connects an external CSS file to this HTML page.
<body>	Contains all visible content — everything the user sees in the browser window.
<script src>	Links an external JavaScript file. Placed at the END of body so HTML loads first.

PART 2 — Core HTML Elements

2.1 Headings — <h1> to <h6>

HTML provides six levels of headings. They define a content hierarchy — like a book with chapters, sections, and sub-sections. Headings are critical for SEO and screen reader navigation.

```
<h1>Main Page Title — There should be only ONE h1 per page</h1>
<h2>Major Section Heading</h2>
<h3>Sub-section Heading</h3>
<h4>Sub-sub-section Heading</h4>
<h5>Rarely used — minor detail heading</h5>
<h6>Smallest heading — used for captions or labels</h6>
```

HEADING RULES — NEVER BREAK THESE

Rule 1: Every page must have exactly ONE <h1> — it is the main title of the page.

Rule 2: Never skip heading levels. Do not jump from <h2> to <h4>.

Rule 3: Choose headings for MEANING and structure — not because of their size.

Use CSS to control size. Use headings to define content hierarchy.

Rule 4: Google reads headings to understand your page content (SEO).

Rule 5: Screen readers use headings to help blind users navigate the page.

2.2 Paragraphs & Text Formatting

```
<p>This is a paragraph. It displays as a block with space above and below.</p>

<strong>Bold — important text (semantic weight)</strong>
<b>Bold — visually bold only (no semantic meaning)</b>

<em>Italic — emphasised text (semantic stress)</em>
<i>Italic — visually italic only (no semantic meaning)</i>

<u>Underlined text</u>
<s>Strikethrough — text that is no longer accurate</s>

<mark>Highlighted text</mark>
<small>Small print — legal disclaimers, copyright notices</small>
<sub>Subscript — e.g. H<sub>2</sub>O</sub>
<sup>Superscript — e.g. E = mc<sup>2</sup></sup>
<code>Inline code — e.g. use the console.log() function</code>
<br> <!-- Line break — use sparingly, prefer CSS margin/padding -->
<hr> <!-- Horizontal rule — a thematic break between content sections -->
```

strong vs b | em vs i — Know the Difference

 and carry SEMANTIC meaning — they tell screen readers and search engines that the text is important or stressed. Screen readers may change their voice tone.

 and <i> are PURELY VISUAL — they look bold or italic but carry no extra meaning.

Best practice: Always prefer and over and <i>.

2.3 Links — <a>

The anchor tag <a> creates hyperlinks — the foundation of the web. Every link on the internet is an <a> element.

```
<!-- External link - opens another website -->
<a href="https://google.com">Go to Google</a>

<!-- Internal link - navigates to another page in your own site -->
<a href="about.html">About Us</a>

<!-- Anchor link - jumps to a section on the same page -->
<a href="#contact">Jump to Contact Section</a>
<section id="contact">...</section>

<!-- Open in a new tab -->
<a href="https://google.com" target="_blank" rel="noopener noreferrer">Open in new
tab</a>

<!-- Email link -->
<a href="mailto:lucky@example.com">Email Us</a>

<!-- Phone link (clickable on mobile) -->
<a href="tel:+254712345678">Call Us</a>
```

<a> Attribute	Purpose
href	Hypertext REference — the URL or path the link points to. Required.
target="_blank"	Opens the link in a new browser tab.
rel="noopener"	Security attribute — prevents the new tab from accessing your page. Always use with target="_blank".
title	A tooltip that appears when the user hovers over the link.
download	Tells the browser to download the linked file instead of opening it.

2.4 Images —

Images are self-closing elements. They pull in an image file and display it on the page.

```
<!-- Basic image -->


<!-- Image with width and height -->


<!-- Image from an external URL -->


<!-- Image inside a figure with a caption -->
<figure>
  
  <figcaption>The Get Techy With Lucky team — Nyeri, 2025</figcaption>
</figure>
```

THE alt ATTRIBUTE IS NOT OPTIONAL

The alt (alternative text) attribute describes what the image shows.

It is displayed if the image fails to load.

It is read aloud by screen readers for visually impaired users.

It is indexed by Google Images and improves SEO.

Rule: If the image is meaningful — describe it. If decorative only — use alt="".

BAD: alt="image1.jpg" | GOOD: alt="Lucky Nakola teaching HTML in a classroom"

2.5 Lists

HTML provides three types of lists, each serving a different purpose.

```
<!-- Unordered list — bullet points, order does not matter -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- Ordered list — numbered, order matters -->
<ol>
  <li>Open VS Code</li>
  <li>Create index.html</li>
  <li>Write your HTML</li>
  <li>Open in browser</li>
</ol>

<!-- Description list — term and definition pairs -->
<dl>
```

```
<dt>HTML</dt>
<dd>HyperText Markup Language – the structure of a webpage.</dd>
<dt>CSS</dt>
<dd>Cascading Style Sheets – the styling of a webpage.</dd>
</dl>

<!-- Nested list – a list inside a list -->
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

2.6 Tables

Tables are used to display data that has a natural row-and-column structure — like a timetable, a price list, or a comparison chart. Never use tables for layout.

```
<table>
  <caption>Web Development Languages</caption>

  <thead>
    <tr>
      <th scope="col">Language</th>
      <th scope="col">Role</th>
      <th scope="col">Type</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>HTML</td>
      <td>Structure</td>
      <td>Markup</td>
    </tr>
    <tr>
      <td>CSS</td>
      <td>Styling</td>
      <td>Style Sheet</td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <td colspan="3">All three are essential for frontend development.</td>
    </tr>
```

```
</tfoot>
```

```
</table>
```

Table Element	Purpose
<table>	The container for the entire table.
<caption>	A visible title for the table — helps with accessibility.
<thead>	Table header group — wraps the top row of column headings.
<tbody>	Table body group — wraps all data rows.
<tfoot>	Table footer group — wraps summary or totals rows.
<tr>	Table Row — a horizontal row.
<th>	Table Header cell — bold and centred by default. Use scope="col" or scope="row".
<td>	Table Data cell — a regular data cell.
colspan	Makes a cell span multiple columns: <td colspan="3">
rowspan	Makes a cell span multiple rows: <td rowspan="2">

2.7 Forms — The Complete Reference

Forms are one of the most important structures in HTML. They collect input from users and send it to a server or handle it with JavaScript.

```
<form action="submit.php" method="POST" novalidate>

  <!-- Text input -->
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" placeholder="Lucky Nakola" required>

  <!-- Email input -->
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <!-- Password input -->
  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="password" minlength="8" required>

  <!-- Number input -->
  <label for="age">Age:</label>
  <input type="number" id="age" name="age" min="1" max="120">

  <!-- Date picker -->
  <label for="dob">Date of Birth:</label>
  <input type="date" id="dob" name="dob">

  <!-- Dropdown select -->
  <label for="country">Country:</label>
  <select id="country" name="country">
    <option value="">-- Select --</option>
    <option value="ke">Kenya</option>
    <option value="ug">Uganda</option>
  </select>

  <!-- Textarea -->
  <label for="msg">Message:</label>
  <textarea id="msg" name="message" rows="5" cols="40"></textarea>

  <!-- Radio buttons -->
  <fieldset>
    <legend>Gender:</legend>
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Male</label>
    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Female</label>
  </fieldset>

  <!-- Checkboxes -->
  <input type="checkbox" id="terms" name="terms" required>
```

```
<label for="terms">I agree to the Terms and Conditions</label>

<!-- File upload -->
<label for="photo">Profile Photo:</label>
<input type="file" id="photo" name="photo" accept="image/*">

<!-- Range slider -->
<label for="rating">Rating: <span id="ratingVal">5</span></label>
<input type="range" id="rating" name="rating" min="1" max="10" value="5">

<!-- Hidden input -->
<input type="hidden" name="page_id" value="42">

<!-- Submit button -->
<button type="submit">Send Message</button>
<button type="reset">Clear Form</button>

</form>
```

Input Types Reference

Input Type	What It Does	Common Use
text	Single-line text entry	Name, username, city
email	Email address with format check	user@example.com
password	Text hidden with dots	Login password
number	Numeric input with min/max	Age, quantity, price
date	Date picker calendar	Date of birth, booking date
time	Time picker	Appointment time
url	URL with format validation	Website address
tel	Phone number input	Mobile number
search	Styled search field	Site search bar
checkbox	Tick one or many options	Agree to terms, interests
radio	Select exactly one from a group	Gender, payment method
file	Opens file picker dialog	Upload photo, attach document
range	Slider from min to max	Rating, volume, brightness
color	Colour picker	Theme preference
hidden	Not visible, sends data silently	CSRF token, page ID
submit	Button that submits the form	Sign Up, Send, Checkout
reset	Button that clears the form	Clear, Start Over

Input Type	What It Does	Common Use
button	Generic button, no default action	Trigger JS function

2.8 Media Elements — Video, Audio & Canvas

```
<!-- HTML5 Video with multiple formats for browser support -->
<video controls width="640" height="360" poster="thumbnail.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <source src="lesson.webm" type="video/webm">
  Your browser does not support the video tag.
</video>

<!-- HTML5 Audio -->
<audio controls>
  <source src="podcast.mp3" type="audio/mpeg">
  Your browser does not support the audio tag.
</audio>

<!-- Embedded external video (YouTube) -->
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  width="560" height="315"
  allowfullscreen
  title="Web Development Tutorial">
</iframe>

<!-- Canvas - JavaScript draws here -->
<canvas id="myCanvas" width="500" height="300"></canvas>
```

Media Attribute	What It Does
controls	Shows play, pause, volume controls to the user.
autoplay	Starts playing automatically (use sparingly — annoying to users).
muted	Starts with audio muted.
loop	Plays the media on repeat indefinitely.
poster	Video only — a thumbnail image shown before play begins.
<source>	Provides multiple media formats so any browser can play one it supports.

PART 3 — HTML Attributes

Attributes provide additional information about HTML elements. They always appear inside the opening tag, always come in name="value" pairs (most of the time), and modify or extend the element's behaviour.

3.1 Global Attributes

Global attributes can be applied to ANY HTML element.

Attribute	Example	Purpose
id	<code>id="hero"</code>	Unique identifier. One per page. Used by CSS (<code>#hero</code>) and JS (<code>getElementById</code>).
class	<code>class="card active"</code>	One or more class names (space-separated). Used by CSS and JS for grouping.
style	<code>style="color:red"</code>	Inline CSS styles — use sparingly, prefer external CSS.
title	<code>title="More info"</code>	Tooltip text shown when the user hovers.
lang	<code>lang="sw"</code>	Declares the language of the element's content.
dir	<code>dir="rtl"</code>	Text direction: <code>ltr</code> (left-to-right) or <code>rtl</code> (right-to-left, e.g. Arabic).
tabindex	<code>tabindex="0"</code>	Controls keyboard tab navigation order.
hidden	<code>hidden</code>	Boolean — hides the element (like <code>display:none</code> but semantic).
draggable	<code>draggable="true"</code>	Makes the element draggable (used with HTML5 Drag and Drop API).
contenteditable	<code>contenteditable</code>	Makes the element's content editable directly in the browser.
data-*	<code>data-user-id="42"</code>	Custom data attributes — store extra info for JavaScript to read.
aria-*	<code>aria-label="Close"</code>	Accessibility attributes for screen readers (covered in Part 4).

3.2 Element-Specific Attributes

Input & Form Attributes

Attribute	Example	Purpose
name	<code>name="email"</code>	The field's name — used to identify the value when the form is submitted.
value	<code>value="Kenya"</code>	Pre-filled value. Also the value sent to server when submitted.
placeholder	<code>placeholder="..."</code>	Ghost text shown when field is empty. Disappears when the user types.
required	<code>required</code>	Boolean — form cannot be submitted if this field is empty.
disabled	<code>disabled</code>	Boolean — field is greyed out and cannot be interacted with.
readonly	<code>readonly</code>	Boolean — user can see but not change the value.
min/max	<code>min="0" max="100"</code>	Minimum and maximum values for number, date, range inputs.
minlength	<code>minlength="8"</code>	Minimum number of characters for text inputs.
maxlength	<code>maxlength="50"</code>	Maximum number of characters allowed.
pattern	<code>pattern="[0-9]{10}"</code>	A regular expression the value must match for validation.
autocomplete	<code>autocomplete="off"</code>	Controls whether the browser suggests previously entered values.
multiple	<code>multiple</code>	Allows multiple file selections or email addresses.
accept	<code>accept="image/*"</code>	File input only — restricts acceptable file types.
autofocus	<code>autofocus</code>	Automatically focuses this field when the page loads.

Link & Image Attributes

Attribute	Example	Purpose
href	<code>href="page.html"</code>	Link destination URL (for <code><a></code>).
target	<code>target="_blank"</code>	Where to open the link: <code>_blank</code> (new tab), <code>_self</code> (same tab), <code>_parent</code> , <code>_top</code> .
rel	<code>rel="noopener"</code>	Relationship between current page and the linked resource. Security-critical with <code>target="_blank"</code> .
src	<code>src="photo.jpg"</code>	Source file path for <code></code> , <code><script></code> , <code><iframe></code> .
alt	<code>alt="Description"</code>	Alternative text for images. Required for accessibility.

Attribute	Example	Purpose
width	width="400"	Element width in pixels.
height	height="300"	Element height in pixels.
loading	loading="lazy"	Defers loading images until they are near the viewport. Improves performance.
srcset	srcset="img@2x.jpg 2x"	Provides multiple image sources for different screen resolutions.

PART 4 — Semantic HTML

4.1 What Is Semantic HTML?

Semantic HTML means choosing HTML elements based on their MEANING — what the content IS — rather than how it looks. A semantic element describes its own purpose clearly, both to the developer reading the code and to the browser, search engine, and assistive technology processing it.

Approach	What It Uses	What Browsers & Tools Understand
Non-Semantic HTML	Uses generic containers with no inherent meaning: <code><div></code> , <code></code> .	The browser knows it is a box. Nothing more.
Semantic HTML	Uses meaningful elements: <code><header></code> , <code><nav></code> , <code><main></code> , <code><article></code> , <code><footer></code> .	The browser knows it is a navigation bar, an article, a footer, etc.

WHY SEMANTIC HTML MATTERS — 4 BIG REASONS

1. **ACCESSIBILITY** — Screen readers use semantic elements to help visually impaired users navigate the page. A `<nav>` tells them 'this is the navigation'. A `<main>` lets them skip straight to the content.
2. **SEO** — Search engines (Google, Bing) use semantic structure to understand your content. An `<article>` tells Google 'this is a piece of content worth indexing'. An `<h1>` tells it 'this is the most important heading on the page'.
3. **MAINTAINABILITY** — Semantic code is much easier to read and maintain. `<header>` vs `<div class="header">` — which is clearer?
4. **BROWSER BEHAVIOUR** — Browsers apply correct default styles and behaviours to semantic elements automatically. They also render them correctly for printing and mobile.

4.2 Non-Semantic vs Semantic — Side by Side

```
<!-- ❌ NON-SEMANTIC — tells us nothing about the content -->
<div class="header">
  <div class="logo">Lucky Nakola</div>
  <div class="nav">
    <div class="nav-item"><a href="home.html">Home</a></div>
    <div class="nav-item"><a href="about.html">About</a></div>
  </div>
</div>
```

```
<!-- ☒ SEMANTIC — perfectly describes what every element IS -->
<header>
  <div class="logo">Lucky Nakola</div>
  <nav aria-label="Main navigation">
    <ul>
      <li><a href="home.html">Home</a></li>
      <li><a href="about.html">About</a></li>
    </ul>
  </nav>
</header>
```

4.3 Page-Level Semantic Elements

These elements define the major structural zones of a webpage.

<header> — Page or Section Header

The <header> element represents the introductory content of a page or section. It typically contains the logo, main navigation, a headline, or a hero area. A page can have multiple <header> elements — one for the page, and one inside each <article> or <section>.

```
<header>
  <a href="/">
    
  </a>
  <nav aria-label="Primary">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/about">About</a></li>
      <li><a href="/services">Services</a></li>
      <li><a href="/contact">Contact</a></li>
    </ul>
  </nav>
  <button class="btn-primary">Get Started</button>
</header>
```

<nav> — Navigation

The <nav> element marks a set of navigation links — menus, breadcrumbs, pagination, or a table of contents. Not every group of links needs a <nav> — only the major navigation blocks. Use the aria-label attribute when you have more than one <nav> on a page to distinguish them.

```
<!-- Primary navigation -->
<nav aria-label="Primary">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/blog">Blog</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>
```

```
<!-- Breadcrumb navigation -->
<nav aria-label="Breadcrumb">
  <ol>
    <li><a href="/">Home</a></li>
    <li><a href="/blog">Blog</a></li>
    <li aria-current="page">HTML Basics</li>
  </ol>
</nav>
```

<main> — Main Content

The <main> element contains the primary content of the page — the content that is unique to this page and not repeated across other pages. There must be only ONE <main> per page. It should never be nested inside <header>, <footer>, <nav>, <aside>, or <article>.

```
<body>
  <header>...</header>
  <nav>...</nav>

  <main>
    <!-- Everything unique to THIS page goes here -->
    <h1>Welcome to Get Techy With Lucky</h1>
    <section>...</section>
    <article>...</article>
  </main>

  <footer>...</footer>
</body>
```

<section> — Thematic Section

The <section> element groups related content that shares a common theme. Each <section> should have its own heading (<h2>, <h3>, etc.). Use <section> when the content belongs together conceptually but is not self-contained enough to be an <article>.

```
<main>
  <section>
    <h2>Our Services</h2>
    <p>We offer professional web development training...</p>
    <!-- Service cards go here -->
  </section>

  <section>
    <h2>What Our Students Say</h2>
    <!-- Testimonials go here -->
  </section>
</main>
```

<article> — Self-Contained Content

The <article> element represents a self-contained piece of content that could be distributed or syndicated independently — a blog post, a news article, a forum post, a product card, a user comment. Ask yourself: 'If I took this content out of the page, would it still make sense on its own?' If yes — it is an <article>.

```
<article>
  <header>
    <h2>Getting Started with HTML in 2025</h2>
    <p>By <address><a rel="author" href="/lucky">Lucky Nakola</a></address></p>
    <time datetime="2025-06-01">1 June 2025</time>
  </header>

  <p>HTML is the foundation of every website...</p>

  <section>
    <h3>Why Learn HTML First?</h3>
    <p>Before CSS or JavaScript, you need structure...</p>
  </section>

  <footer>
    <p>Tags: <a href="/html">HTML</a>, <a href="/web-dev">Web Dev</a></p>
  </footer>
</article>
```

<aside> — Complementary Content

The <aside> element marks content that is tangentially related to the surrounding content — a sidebar, a pull quote, an advertisement, a related articles list, or author bio. When used inside an <article>, it relates to that article. When used directly inside <main>, it relates to the whole page.

```
<!-- Sidebar aside (relates to whole page) -->
<aside aria-label="Related topics">
  <h3>Also Read</h3>
  <ul>
    <li><a href="/css-basics">CSS Foundations</a></li>
    <li><a href="/js-intro">JavaScript for Beginners</a></li>
  </ul>
</aside>

<!-- Inside an article (relates to that article) -->
<article>
  <p>HTML was created in 1991...</p>
  <aside>
    <p><strong>Did you know?</strong> The first website ever built
    is still live at info.cern.ch</p>
  </aside>
</article>
```

<footer> — Page or Section Footer

The <footer> element represents closing content for its nearest ancestor section, article, or the page. For a page footer, it typically contains copyright, links, contact info, and a sitemap. Like <header>, a page can have multiple footers — one per <article> or <section>.

```
<footer>
  <div class="footer-grid">
    <div>
      <h3>Get Techy With Lucky</h3>
      <p>Practical web development training in Kenya.</p>
    </div>
    <nav aria-label="Footer navigation">
      <h3>Quick Links</h3>
      <ul>
        <li><a href="/privacy">Privacy Policy</a></li>
        <li><a href="/terms">Terms of Service</a></li>
      </ul>
    </nav>
    <address>
      <h3>Contact</h3>
      <a href="mailto:lucky@example.com">lucky@example.com</a>
    </address>
  </div>
  <p><small>&copy; 2025 Lucky Nakola. All rights reserved.</small></p>
</footer>
```

4.4 Inline Semantic Text Elements

These elements give meaning to words and phrases within a paragraph.

Element	Meaning	Example
<code><time></code>	Represents a specific date or time	<code><time datetime="2025-12-25">Christmas</time></code>
<code><address></code>	Contact information for nearest ancestor	<code><address>lucky@example.com</address></code>
<code><abbr></code>	Abbreviation with full expansion on hover	<code><abbr title="HyperText Markup Language">HTML</abbr></code>
<code><cite></code>	Title of a creative work (book, film, song)	<code><cite>The Web Design Book</cite></code>
<code><q></code>	Short inline quotation with quotes auto-added	She said <code><q>Learn HTML first</q></code>
<code><blockquote></code>	Long block quotation from another source	<code><blockquote cite="https://...">...</blockquote></code>
<code><dfn></code>	The defining instance of a term	<code><dfn>HTML</dfn></code> stands for...
<code><kbd></code>	Keyboard input the user should type	Press <code><kbd>Ctrl</kbd>+<kbd>S</kbd></code>
<code><samp></code>	Sample output from a program	<code><samp>Error: file not found</samp></code>
<code><var></code>	A variable in math or programming	<code><var>x</var> = <var>y</var> + 2</code>
<code><mark></code>	Highlighted/relevant text (like highlighter)	<code><mark>important term</mark></code>
<code></code>	Content that has been removed/deleted	<code>Old price: KES 5,000</code>
<code><ins></code>	Content that has been added/inserted	<code><ins>New price: KES 3,500</ins></code>
<code><ruby></code>	East Asian pronunciation annotation	<code><ruby>漢<rt>Kan</rt></ruby></code>

4.5 Structural Semantic Elements

`<figure>` and `<figcaption>`

`<figure>` wraps self-contained content that is referenced from the main text — an image, diagram, code snippet, or chart. `<figcaption>` provides a visible caption for it.

```
<figure>
  
  <figcaption>
    Figure 1: The HTML Document Object Model (DOM) tree structure.
  </figcaption>
</figure>
```

```
<!-- figure also works for code blocks -->
<figure>
  <pre><code>
    const greeting = 'Hello, World!';
    console.log(greeting);
  </code></pre>
  <figcaption>Code Example 1: Logging a greeting to the console.</figcaption>
</figure>
```

<details> and <summary>

Creates a native, no-JavaScript accordion. <summary> is the always-visible label. Clicking it expands to reveal the <details> content.

```
<details>
  <summary>What is HTML?</summary>
  <p>HTML stands for HyperText Markup Language. It is the standard
  language for creating web pages and provides the structure for
  all content on the internet.</p>
</details>

<!-- details open by default -->
<details open>
  <summary>Course Requirements</summary>
  <ul>
    <li>A computer or smartphone</li>
    <li>A text editor (VS Code)</li>
    <li>A web browser (Google Chrome)</li>
  </ul>
</details>
```

<dialog>

The <dialog> element represents a modal or popup dialog box. It is controlled with JavaScript using dialog.showModal() and dialog.close(). A native HTML alternative to custom modal implementations.

```
<dialog id="myModal">
  <h2>Confirm Action</h2>
  <p>Are you sure you want to delete this item?</p>
  <button onclick="document.getElementById('myModal').close()">Cancel</button>
  <button>Confirm Delete</button>
</dialog>

<button onclick="document.getElementById('myModal').showModal()">
  Open Dialog
</button>
```

<progress> and <meter>

```
<!-- progress - shows completion of a task (like a loading bar) -->
<label for="upload">Uploading file:</label>
<progress id="upload" value="65" max="100">65%</progress>

<!-- meter - shows a scalar measurement within a known range -->
<label for="score">Quiz Score:</label>
<meter id="score" value="78" min="0" max="100" low="40" high="70" optimum="90">
  78 out of 100
</meter>
```

4.6 Complete Semantic Elements Reference Table

Semantic Element	Purpose & When to Use It
<header>	Introductory content for a page or section — logo, nav, hero.
<nav>	Major navigation block — primary menu, breadcrumbs, pagination.
<main>	The primary, unique content of the page. One per page only.
<section>	A thematic grouping of related content, each with its own heading.
<article>	Self-contained, independently distributable content.
<aside>	Tangentially related content — sidebar, callout, related links.
<footer>	Closing content for a page or section — copyright, links, contact.
<figure>	Self-contained content (image, code, chart) referenced from text.
<figcaption>	A visible caption for a <figure> element.
<time>	A specific point in time or a duration. Use datetime attribute.
<address>	Contact information for the nearest ancestor article/body.
<blockquote>	A long quotation from an external source. Use cite attribute.
<q>	A short inline quotation. Browser adds quotation marks.
<abbr>	An abbreviation or acronym. Use title for the full expansion.
<cite>	The title of a creative work being referenced or quoted.
<details>	A disclosure widget — collapsed by default, expanded on click.
<summary>	The visible heading/label for a <details> element.
<dialog>	A modal or popup dialog box. Controlled via JavaScript.
<progress>	Shows the completion progress of a task (0–100).
<meter>	Shows a scalar value within a known range (with low/high/optimum).
<mark>	Highlighted text — relevant to the user's search or context.
<dfn>	The defining instance of a term being introduced.

Semantic Element	Purpose & When to Use It
<kbd>	Keyboard input — keys the user should press.
<samp>	Sample output from a computer program.
<var>	A variable in mathematical or programming context.
	Content that has been deleted or removed.
<ins>	Content that has been inserted or added.

PART 5 — Real-World Semantic Page Structure

This is how a complete, professional, semantic HTML page looks. Study this structure carefully — it is the template you should start every project with.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Learn web development with Lucky Nakola in Kenya.">
  <title>Get Techy With Lucky – Web Development Training</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <!-- === SITE HEADER === -->
  <header>
    <a href="/"></a>
    <nav aria-label="Primary">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about">About</a></li>
        <li><a href="/courses">Courses</a></li>
        <li><a href="/blog">Blog</a></li>
        <li><a href="/contact">Contact</a></li>
      </ul>
    </nav>
    <a href="/enroll" class="btn">Enroll Now</a>
  </header>

  <!-- === MAIN CONTENT === -->
  <main>

    <!-- Hero Section -->
    <section class="hero" aria-labelledby="hero-heading">
      <h1 id="hero-heading">Learn Web Development the Right Way</h1>
      <p>Practical, beginner-friendly HTML, CSS, JavaScript, PHP and MySQL training.</p>
      <a href="/courses" class="btn-primary">Browse Courses</a>
      <a href="/about" class="btn-secondary">Meet Lucky</a>
    </section>

    <!-- Services Section -->
    <section aria-labelledby="services-heading">
      <h2 id="services-heading">What We Teach</h2>
```

```
<div class="services-grid">
  <article class="service-card">
    <h3>HTML & CSS</h3>
    <p>Build the structure and style of modern, responsive websites.</p>
  </article>
  <article class="service-card">
    <h3>JavaScript</h3>
    <p>Add interactivity, validate forms, and build dynamic SPAs.</p>
  </article>
  <article class="service-card">
    <h3>PHP & MySQL</h3>
    <p>Build backend logic and database-driven web applications.</p>
  </article>
</div>
</section>

<!-- Blog Section -->
<section aria-labelledby="blog-heading">
  <h2 id="blog-heading">Latest Articles</h2>

  <article>
    <header>
      <h3><a href="/blog/html-guide">The Complete HTML Master Guide</a></h3>
      <p>By <address><a rel="author" href="/about">Lucky Nakola</a></address>
        &bull; <time datetime="2025-06-01">1 June 2025</time></p>
    </header>
    <p>Everything you need to know about HTML and Semantic HTML...</p>
    <a href="/blog/html-guide">Read More &rarr;</a>
  </article>

</section>

<!-- FAQ Section -->
<section aria-labelledby="faq-heading">
  <h2 id="faq-heading">Frequently Asked Questions</h2>
  <details>
    <summary>Do I need any prior experience to join?</summary>
    <p>No! Our courses start from absolute zero. All you need is a computer
      and the motivation to learn.</p>
  </details>
  <details>
    <summary>Are the sessions in person or online?</summary>
    <p>We offer both in-person sessions in Nyeri and live online classes.</p>
  </details>
</section>

</main>

<!-- == SIDEBAR == -->
<aside aria-label="Newsletter signup">
```

```
<h2>Stay Updated</h2>
<p>Get free HTML tips and tutorials delivered to your inbox.</p>
<form action="/subscribe" method="POST">
  <label for="sub-email">Email Address:</label>
  <input type="email" id="sub-email" name="email" required>
  <button type="submit">Subscribe Free</button>
</form>
</aside>

<!-- === SITE FOOTER === -->
<footer>
  <nav aria-label="Footer">
    <ul>
      <li><a href="/privacy">Privacy Policy</a></li>
      <li><a href="/terms">Terms of Service</a></li>
      <li><a href="/sitemap">Sitemap</a></li>
    </ul>
  </nav>
  <address>
    <a href="mailto:lucky@example.com">lucky@example.com</a> &bull;
    <a href="tel:+254712345678">+254 712 345 678</a>
  </address>
  <p><small>&copy; 2025 Get Techy With Lucky. All rights reserved.</small></p>
</footer>

<script src="app.js"></script>
</body>
</html>
```

PART 6 — HTML Best Practices & Accessibility

6.1 HTML Best Practices

Best Practice	Why and How
Use lowercase tags	Always write tags in lowercase: <code><section></code> not <code><SECTION></code> .
Close all tags	Close every tag that is not self-closing: <code></p></code> , <code></code> , <code></div></code> .
Quote all attributes	Always wrap attribute values in quotes: <code>href="page.html"</code> not <code>href=page.html</code> .
Use semantic elements	Prefer <code><header></code> , <code><nav></code> , <code><main></code> over <code><div class="header"></code> , etc.
One <code><h1></code> per page	There must be exactly one <code><h1></code> — it is the main title of the page.
Never skip heading levels	Go <code>h1</code> → <code>h2</code> → <code>h3</code> . Never <code>h1</code> → <code>h3</code> . Headings define document structure.
Always alt on images	Every <code></code> needs an alt attribute. Empty <code>alt=""</code> for decorative images.
Label every input	Use <code><label for="id"></code> linked to the input's id. Never use placeholder alone.
Method POST for sensitive data	Login, register, payment — always use <code>method="POST"</code> .
Validate your HTML	Use validator.w3.org to check your HTML for errors regularly.
Indent consistently	Use 2-space or 4-space indentation. Nested elements should be indented.
Comment your code	Add <code><!-- comments --></code> to explain sections, especially for team projects.
Separate concerns	HTML = structure. CSS = style. JS = behaviour. Keep them in separate files.
No inline styles	Avoid <code>style="..."</code> in HTML. Write CSS in external stylesheets.
Use <code>loading="lazy"</code> on images	Speeds up page load by deferring off-screen image loading.
Add lang to <code><html></code>	<code>lang="en"</code> helps search engines and screen readers identify the page language.

6.2 Accessibility (a11y) — Writing HTML for Everyone

Approximately 1 in 6 people worldwide has some form of disability. Accessible HTML ensures your website works for everyone — including people using screen readers, keyboard-only navigation, or low-vision assistive tools.

ARIA — Accessible Rich Internet Applications

ARIA attributes add accessibility information that HTML alone cannot provide. Use them to enhance — not replace — semantic HTML.

ARIA Attribute	What It Does & Example
aria-label	Provides an accessible name for an element when no visible text label exists. <code><button aria-label="Close dialog">X</button></code>
aria-labelledby	Points to the ID of another element that serves as the label. <code><section aria-labelledby="services-h2"></code>
aria-describedby	Points to an element that provides a longer description.
aria-hidden	Hides an element from screen readers (for decorative icons). <code>★</code>
aria-expanded	Indicates whether a collapsible element is open (true) or closed (false).
aria-current	Marks the current item in a set. <code>aria-current="page"</code> on the active nav link.
aria-live	Announces dynamic content changes to screen readers. Values: polite, assertive.
role	Overrides or supplements the element's implicit role. Only use when semantic HTML cannot.

Keyboard Navigation

- ❖ All interactive elements (links, buttons, inputs) must be focusable via the Tab key.
- ❖ Never remove the default outline on focus with `outline:none` — it helps keyboard users.
- ❖ Use `tabindex="0"` to make non-interactive elements focusable when needed.
- ❖ Use `tabindex="-1"` to remove an element from the tab order programmatically.
- ❖ All JavaScript interactions (click events) should also respond to the Enter/Space key.

Colour & Contrast

- Normal text must have a contrast ratio of at least 4.5:1 against its background.
- Large text (18pt+ or 14pt bold) needs at least 3:1 contrast.
- Never rely on colour alone to convey meaning — add text, icons, or patterns too.
- Test contrast with free tools: WebAIM Contrast Checker, or Chrome DevTools.

QUICK ACCESSIBILITY CHECKLIST

- ☐ Every image has a meaningful alt attribute (or alt="" if purely decorative)
- ☐ Every form input has an associated <label>
- ☐ The page has exactly one <h1> and headings are in logical order
- ☐ Semantic elements are used instead of generic <div> wherever possible
- ☐ All links have descriptive text (not 'click here' or 'read more')
- ☐ Colour contrast meets WCAG AA standards (4.5:1 for normal text)
- ☐ The page can be fully navigated using only the keyboard
- ☐ lang attribute is set on the <html> element
- ☐ The page title is descriptive and unique
- ☐ No content relies on colour alone to convey meaning

PART 7 — Comprehensive Quiz

Test your understanding of everything covered in this guide. Attempt all sections independently before checking your answers with the class.

Section A — Multiple Choice (Circle the correct answer — 1 mark each)

1. What does HTML stand for?

- a) High Tech Markup Language
- b) HyperText Markup Language
- c) HyperText Machine Language
- d) Home Tool Markup Language

2. Which of the following correctly describes what semantic HTML means?

- a) HTML that only uses `<div>` and `<span>` elements
- b) HTML that uses elements based on what the content IS, not just how it looks
- c) HTML that uses inline styles for better visual design
- d) HTML that is minified to reduce file size

3. Which element should appear ONLY ONCE on any given HTML page?

- a) `<section>`
- b) `<article>`
- c) `<main>`
- d) `<aside>`

4. What is the correct HTML5 document type declaration?

- a) `<!HTML>`
- b) `<DOCTYPE html5>`
- c) `<!DOCTYPE html>`
- d) `<?xml version="1.0">`

5. Which element is MOST appropriate for marking the primary navigation menu of a website?

- a) `<menu>`
- b) `<div class="nav">`
- c) `<nav>`
- d) `<ul class="navigation">`

6. An image of a decorative background pattern has no meaningful content. What should its alt attribute be?

- a) `alt="background pattern"`
- b) `alt="image"`
- c) `alt=""`
- d) The alt attribute should be omitted entirely

7. Which pair of elements creates a native, JavaScript-free accordion/disclosure widget?

- a) `<accordion>` and `<item>`
- b) `<collapse>` and `<expand>`
- c) `<details>` and `<summary>`
- d) `<section>` and `<aside>`

8. What is the purpose of the `<time>` element's datetime attribute?

- a) To style the time display using CSS
- b) To provide a machine-readable date/time format for browsers and search engines
- c) To set an alarm that triggers a JavaScript function
- d) To display a live clock on the page

9. What is the key difference between `<strong>` and `<b>`?

- a) `<strong>` makes text bigger; `<b>` makes it bold
- b) `<strong>` carries semantic importance; `<b>` is purely visual styling

- c) `<b>` is for buttons; `<strong>` is for strong passwords
- d) There is no difference — they produce identical output
10. Which HTML element should be used to wrap content such as an image and its caption together?
- a) `<picture>` b) `<figure>` c) `<div>` d) `<gallery>`
11. What does the `aria-label` attribute do?
- a) Applies a CSS class to an element
- b) Provides an accessible name for an element when no visible text label exists
- c) Links the element to an external ARIA stylesheet
- d) Marks the element as the main landmark of the page
12. In an HTML table, which element groups the column heading row?
- a) `<thead>` b) `<th>` c) `<header>` d) `<colgroup>`
13. Which input type creates a calendar date picker?
- a) `type="calendar"` b) `type="picker"` c) `type="date"` d) `type="datetime"`
14. What is the purpose of the `<meta name="viewport">` tag?
- a) It sets the background colour of the page on mobile devices
- b) It controls how the page scales and sizes itself on different screen sizes
- c) It defines the maximum number of pixels the page can render
- d) It prevents users from zooming in on mobile
15. Which element is BEST for wrapping a self-contained blog post?
- a) `<section>` b) `<div>` c) `<article>` d) `<main>`
16. What should you use to link an external CSS file to an HTML document?
- a) `<style href="style.css">` b) `<css src="style.css">`
- c) `<link rel="stylesheet" href="style.css">` d) `<import file="style.css">`
17. Which is the correct way to create a link that opens in a new browser tab?
- a) `<a href="url" open="new">` b) `<a href="url" target="_blank" rel="noopener">`
- c) `<a href="url" window="new">` d) `<a href="url" tab="true">`
18. What is the purpose of the `<fieldset>` and `<legend>` elements in a form?
- a) To style form inputs with a bordered box
- b) To group related form controls with an accessible label for the group
- c) To submit multiple forms at the same time
- d) To mark required fields with an asterisk (*) automatically
19. Which of the following is a SELF-CLOSING HTML element?
- a) `<p>` b) `<div>` c) `<section>` d) `<br>`
20. Which element correctly marks contact information for the page author?
- a) `<contact>` b) `<address>` c) `<info>` d) `<p class="contact">`

Section B — Short Answer (Write 3–5 sentences per answer)

1. Explain the difference between `<section>` and `<article>`. Give a real-world example of when you would use each.
2. What is the DOM? Explain in your own words how the browser uses it when it reads an HTML file.
3. List FIVE reasons why using semantic HTML is better than using only `<div>` and `<span>` elements.
4. Explain what accessibility means in the context of web development and name THREE specific HTML techniques that make a webpage more accessible.
5. What is the difference between the `id` attribute and the `class` attribute? Give an example of when you would use each one.
6. Explain the purpose of the `<head>` section in an HTML document. List at least FIVE elements or attributes that belong in the `<head>` and explain what each one does.
7. Describe the correct use of heading levels (`h1`–`h6`) on a webpage. Why should you never skip a heading level? What are the consequences for SEO and accessibility?
8. Explain the difference between GET and POST form methods. When should you use each one, and why should you never use GET for a login form?
9. What are `data-*` attributes? Give an example of how a developer might use a data attribute on an HTML element and then read it with JavaScript.
10. Describe what the `aria-labelledby` attribute does. Write an example using a `<section>` and an `<h2>` heading where `aria-labelledby` is correctly applied.

Section C — Coding Challenges (Write correct, well-structured HTML)

Challenge 1:

Build a complete, semantic HTML blog article page about any topic of your choice. Your page must include: a `<header>` with a site logo and primary `<nav>`, a `<main>` element containing one `<article>` with its own `<header>` (title, author using `<address>`, and `<time>`), at least two `<section>` elements inside the article each with an h3 heading, one `<figure>` with an image and `<figcaption>`, one `<blockquote>` with a cite attribute, a `<footer>` inside the article, an `<aside>` outside the article with related links, and a site `<footer>` with copyright and a contact `<address>`. Use correct semantic markup throughout.

Challenge 2:

Build a complete accessible HTML registration form for a web development course. The form must include: Full Name (text, required), Email Address (email, required), Phone Number (tel), Password (password, minlength 8, required), Confirm Password (password, required), Course Level dropdown (Beginner / Intermediate / Advanced), At least two checkboxes for areas of interest (HTML & CSS / JavaScript / PHP & MySQL / UI/UX Design), Radio buttons for preferred learning mode (In-Person / Online / Both), a File upload for a profile photo (accept images only), a Textarea for a short bio (max 300 characters), a Terms checkbox (required), and a Submit button. All inputs must have properly associated `<label>` elements. Group the radio buttons and checkboxes using `<fieldset>` and `<legend>`. Add appropriate placeholder text and required attributes.

Section A — Answer Key

Multiple Choice Answers

1. b) HyperText Markup Language
2. b) HTML that uses elements based on what the content IS
3. c) `<main>`
4. c) `<!DOCTYPE html>`
5. c) `<nav>`
6. c) `alt=""` (empty string for decorative images)
7. c) `<details>` and `<summary>`
8. b) Machine-readable format for browsers and search engines
9. b) `<strong>` carries semantic importance; `<b>` is purely visual
10. b) `<figure>`
11. b) Provides an accessible name when no visible text label exists
12. a) `<thead>`
13. c) `type="date"`
14. b) Controls how the page scales on different screen sizes
15. c) `<article>`
16. c) `<link rel="stylesheet" href="style.css">`

- 17. b) `<a href="url" target="_blank" rel="noopener">`
- 18. b) Groups related controls with an accessible label
- 19. d) `<br>`
- 20. b) `<address>`

Write code every day. Read code every day. You will be amazed at how fast you grow.

Get Techy With Lucky | Tech Pulse Insider | Instructor: Lucky Nakola